Lab 21

Building a CI/CD Pipeline with Git using API Controller

WSO2 API Manager 4.7.0 · Practical exercise

Before You Begin

URLs and paths in this guide use placeholders. Replace them with your own values:

- **`<APIM_HOST>`** – host name or IP address of the API Manager server (localhost when API Manager runs on your own laptop).
- **`<HTTPS_PORT>`** – HTTPS port of the dev API Manager: 9443 plus any port offset.
- **`<PROD_HTTPS_PORT>`** – HTTPS port of the prod API Manager (9445 with port offset 2).
- **`<APICTL_DIR>`** – the folder that contains the apictl program after you extract it, for example ~/apictl (Windows: C:\apictl).
- **`<WORK_DIR>`** – a working folder for this lab, for example ~/cicd-lab (Windows: C:\cicd-lab).

The servers use a self-signed certificate by default. The -k option in the apictl commands skips the certificate check for that reason.

Training Objective

Learn how to build a CI/CD flow for APIs with Git and the API Controller (apictl): export an API from a lower environment, keep it and its environment-specific settings in Git, and deploy it to an upper environment with apictl vcs commands.

Business Scenario

PizzaShack keeps its API definitions in Git and wants each change to be promoted from the development environment to production by a pipeline, with production-specific values (such as back-end URLs) applied automatically.

High-Level Steps

- Install and initialise apictl
- Prepare the dev and prod environments in apictl
- Create and publish an API in dev
- Export the API and store it in two Git repositories (Source and Deployment)
- Deploy the API to prod with apictl vcs deploy

Prerequisites

- Two API Manager 4.7.0 environments: **dev** and **prod**. Use the two instances from the Migrating APIs lab (dev on port 9443, prod on port 9445 with offset 2), or work in pairs with one laptop as dev and the other as prod.
- Git installed. apictl is installed in Part 1.

CC BY 4.0

Part 1 – Install and Initialise apictl

1. Download the API Controller (apictl) 4.7.x for your operating system from the WSO2 API Manager **Tooling** download page, and extract it. You get a folder named `apictl` that contains the `apictl` program (Windows: `apictl.exe`).

   macOS / Linux (for example darwin-arm64 for an Apple silicon Mac)

   ```
   tar -xzf apictl-4.7.<x>-<os>-<arch>.tar.gz -C ~/
   ```

   On Windows, right-click the downloaded zip, select **Extract All** and extract it to `C:\`. This gives `C:\apictl`.

2. Add the apictl folder to your PATH so that the `apictl` command works from any folder. Without this step the terminal shows `command not found: apictl`.

   macOS (zsh)

   ```
   echo 'export PATH="$PATH:<APICTL_DIR>"' >> ~/.zshrc
   source ~/.zshrc
   ```

   Linux (bash)

   ```
   echo 'export PATH="$PATH:<APICTL_DIR>"' >> ~/.bashrc
   source ~/.bashrc
   ```

   Windows (Command Prompt) – then close it and open a new Command Prompt

   ```
   setx PATH "%PATH%;C:\apictl"
   ```

   On macOS, if a security message blocks apictl the first time, open **System Settings → Privacy & Security** and click **Allow Anyway**, or run `xattr -d com.apple.quarantine <APICTL_DIR>/apictl`.

3. Initialise apictl by running it once. The first run creates the apictl configuration folder `~/.wso2apictl` (Windows: `%USERPROFILE%\.wso2apictl`), where environments and exported APIs are stored.

   ```
   apictl version
   apictl get envs
   ```

   Expected output: the version (4.7.x) and an empty environment table with only the column headings:

   ```
   Version: 4.7.2
   NAME   API MANAGER ENDPOINT   REGISTRATION ENDPOINT   TOKEN ENDPOINT   ...
   ```

Part 2 – Prepare the Environments

4. Add the two environments to apictl. Skip any that already exist (apictl get envs shows them).

   ```
   apictl add env dev --apim https://<APIM_HOST>:<HTTPS_PORT>
   apictl add env prod --apim https://<APIM_HOST>:<PROD_HTTPS_PORT>
   ```

CC BY 4.0

5. Log in to both environments. apictl asks for the admin password each time.

```
apictl login dev -u admin -k
apictl login prod -u admin -k
```

Part 3 – Create and Publish the API in Dev

6. In the dev Publisher, click **Create API** → **Import Open API**, select **OpenAPI URL**, enter the URL below and click **Next**.

   Dev Publisher

   ```
   https://<APIM_HOST>:<HTTPS_PORT>/publisher
   ```

   OpenAPI URL

   ```
   https://petstore3.swagger.io/api/v3/openapi.json
   ```

7. Enter these values and click **Create**:

Field	Value
Name	`SwaggerPetstore`
Context	`/petstore`
Version	`1.0.0`

8. Under **Endpoints**, set the back-end URLs and save:

Endpoint	URL
Production	`https://dev.wso2.com`
Sandbox	`https://dev.sandbox.wso2.com`

9. Under **Subscriptions**, select the **Gold** business plan and save. Then deploy the API to the **Default** gateway and publish it.

Part 4 – Export the API and Prepare the Git Repositories

10. Export the latest revision of the API from dev:

    ```
    apictl export api -e dev -n SwaggerPetstore -v 1.0.0 --provider admin --latest -k
    ```

    The zip is saved as `~/.wso2apictl/exported/apis/dev/SwaggerPetstore_1.0.0.zip` (Windows: in `%USERPROFILE%\.wso2apictl\exported\apis\dev`).

11. Create the two folders `Source` and `Deployment` and make each a Git repository:

    macOS / Linux

    ```
    mkdir -p <WORK_DIR>/Source <WORK_DIR>/Deployment
    cd <WORK_DIR>/Source && git init
    cd <WORK_DIR>/Deployment && git init
    ```

CC BY 4.0

Windows (Command Prompt)

```
mkdir <WORK_DIR>\Source <WORK_DIR>\Deployment
cd <WORK_DIR>\Source && git init
cd <WORK_DIR>\Deployment && git init
```

If Git asks who you are when you commit later, set your identity once:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

12. Extract the exported zip into `Source`. You get the folder `Source/SwaggerPetstore-1.0.0`.

macOS / Linux

```
unzip ~/.wso2apictl/exported/apis/dev/SwaggerPetstore_1.0.0.zip -d <WORK_DIR>/Source
```

Windows (Command Prompt)

```
tar -xf ^
  %USERPROFILE%\.wso2apictl\exported\apis\dev\SwaggerPetstore_1.0.0.zip ^
  -C <WORK_DIR>\Source
```

13. Generate the deployment folder for the project:

macOS / Linux

```
apictl gen deployment-dir \
  -s <WORK_DIR>/Source/SwaggerPetstore-1.0.0 \
  -d <WORK_DIR>/Deployment
```

Windows (Command Prompt)

```
apictl gen deployment-dir ^
  -s <WORK_DIR>\Source\SwaggerPetstore-1.0.0 ^
  -d <WORK_DIR>\Deployment
```

This creates `Deployment/DeploymentArtifacts_SwaggerPetstore-1.0.0` with `api_meta.yaml`, a `certificates` folder and `params.yaml`.

14. The generated `params.yaml` is a template with every field empty. Open it and replace its whole content with the back-end URLs for each environment. Keep the spaces exactly as shown; YAML uses indentation.

Deployment/DeploymentArtifacts_SwaggerPetstore-1.0.0/params.yaml

```
environments:
  - name: dev
    configs:
      endpoints:
        production:
          url: "https://dev.wso2.com"
        sandbox:
          url: "https://dev.sandbox.wso2.com"
  - name: prod
    configs:
      endpoints:
        production:
          url: "https://prod.wso2.com"
        sandbox:
          url: "https://prod.sandbox.wso2.com"
```

CC BY 4.0

15. Commit both repositories:

```
cd <WORK_DIR>/Source && git add -A && git commit -m "SwaggerPetstore"
cd <WORK_DIR>/Deployment && git add -A && git commit -m "SwaggerPetstore"
```

Part 5 – Deploy to Prod with apictl vcs

16. Tell apictl where the two repositories are:

```
apictl set --vcs-source-repo-path <WORK_DIR>/Source
apictl set --vcs-deployment-repo-path <WORK_DIR>/Deployment
```

17. Initialise each repository for apictl once, and commit the `vcs.yaml` file it creates:

```
cd <WORK_DIR>/Source
apictl vcs init
git add vcs.yaml && git commit -m "apictl vcs init"

cd <WORK_DIR>/Deployment
apictl vcs init
git add vcs.yaml && git commit -m "apictl vcs init"
```

18. In the `Source` folder, check what will be deployed to prod:

```
cd <WORK_DIR>/Source
apictl vcs status -e prod
```

Expected output (shortened):

```
Projects to Deploy (1)
...
1: [save] SwaggerPetstore-1.0.0
```

19. Deploy:

```
apictl vcs deploy -e prod -k
```

Expected: `Successfully imported API`. Running the same command again prints `Everything is up-to-date`, because nothing has changed since the last deployment.

20. Check the result in the prod Publisher. **SwaggerPetstore** is published with the Gold plan, and its endpoints are the prod values from `params.yaml`:

Prod Publisher

```
https://<APIM_HOST>:<PROD_HTTPS_PORT>/publisher
```

Endpoint	Expected URL in prod
Production	`https://prod.wso2.com`
Sandbox	`https://prod.sandbox.wso2.com`

CC BY 4.0

21. Optional: change something (for example a description in the project, or a URL in `params.yaml`), commit it, and run `vcs status` and `vcs deploy` again. Only the changed project is deployed.

    In a real pipeline, a CI server (for example Jenkins or GitHub Actions) runs these two commands on every commit.

Expected Outcome

An API created in dev is stored in Git as an apictl project with environment-specific parameters, and is deployed to prod with apictl vcs deploy, using the prod back-end URLs.

CC BY 4.0